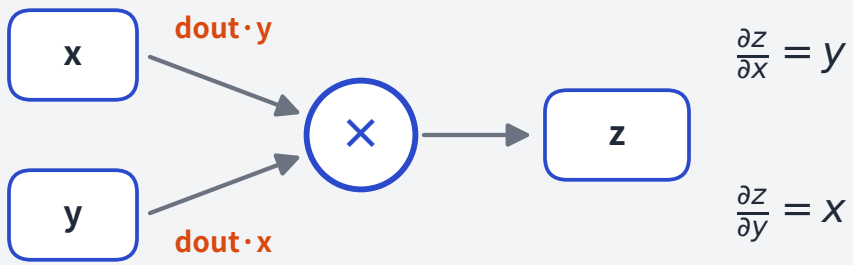
곱셈 계층 (MulLayer)

순전파 때의 입력을 기억해 두었다가, 역전파에서 서로 바꿔 곱한다



`__init__()`

인스턴스 변수 x, y를 초기화 (순전파 값 보관용)

`forward()`

x, y를 받아 곱해서 반환 (out = x * y)

`backward()`

dout에 순전파 값을 서로 바꿔 곱해 하류로 전달

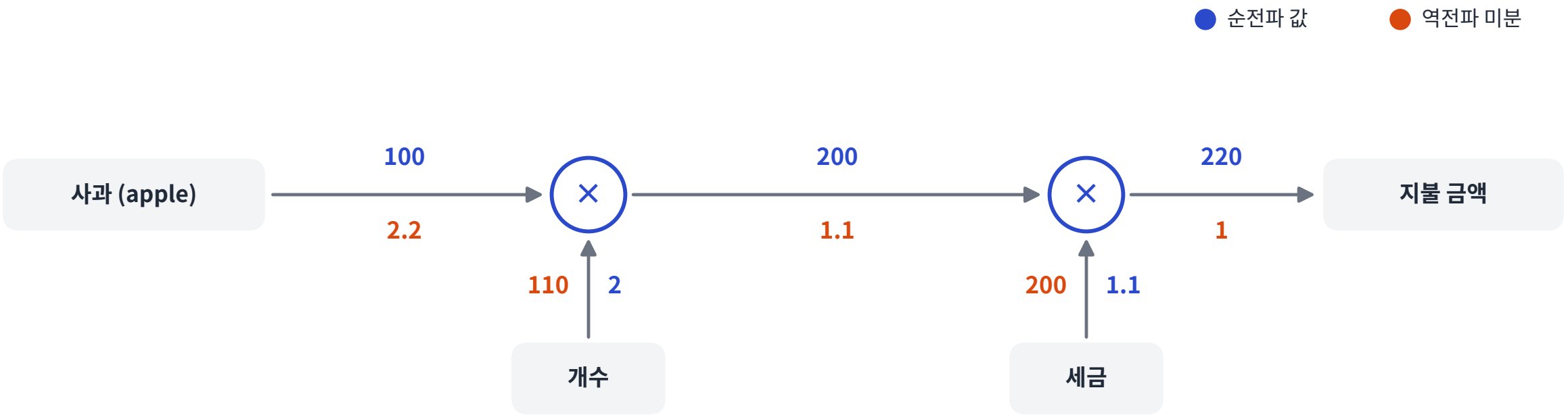
```
class MulLayer:
    def __init__(self):
        self.x = None
        self.y = None

    def forward(self, x, y):
        self.x = x
        self.y = y
        out = x * y
        return out

    def backward(self, dout):
        dx = dout * self.y  # swap x and y
        dy = dout * self.x
        return dx, dy
```

곱셈 계층 활용: 사과 2개 구입

사과 100원 × 2개, 소비세 1.1 → 지불 금액과 각 변수에 대한 미분




```
print(price)  # 220

print(dapple, dapple_num, dtax)  # 2.2  110  200
```

dprice = 1 에서 시작해 거꾸로 전달

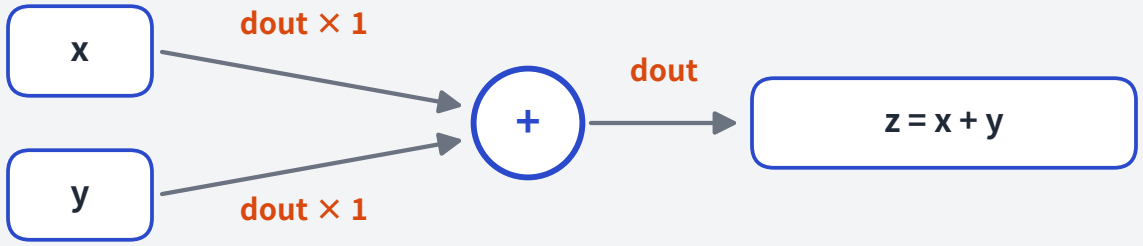
덧셈 계층 (AddLayer)

덧셈 노드의 역전파는 상류 값에 1을 곱해 그대로 흘려보낸다

```
class AddLayer:
    def __init__(self):
        pass

    def forward(self, x, y):
        out = x + y
        return out

    def backward(self, dout):
        dx = dout * 1
        dy = dout * 1
        return dx, dy
```

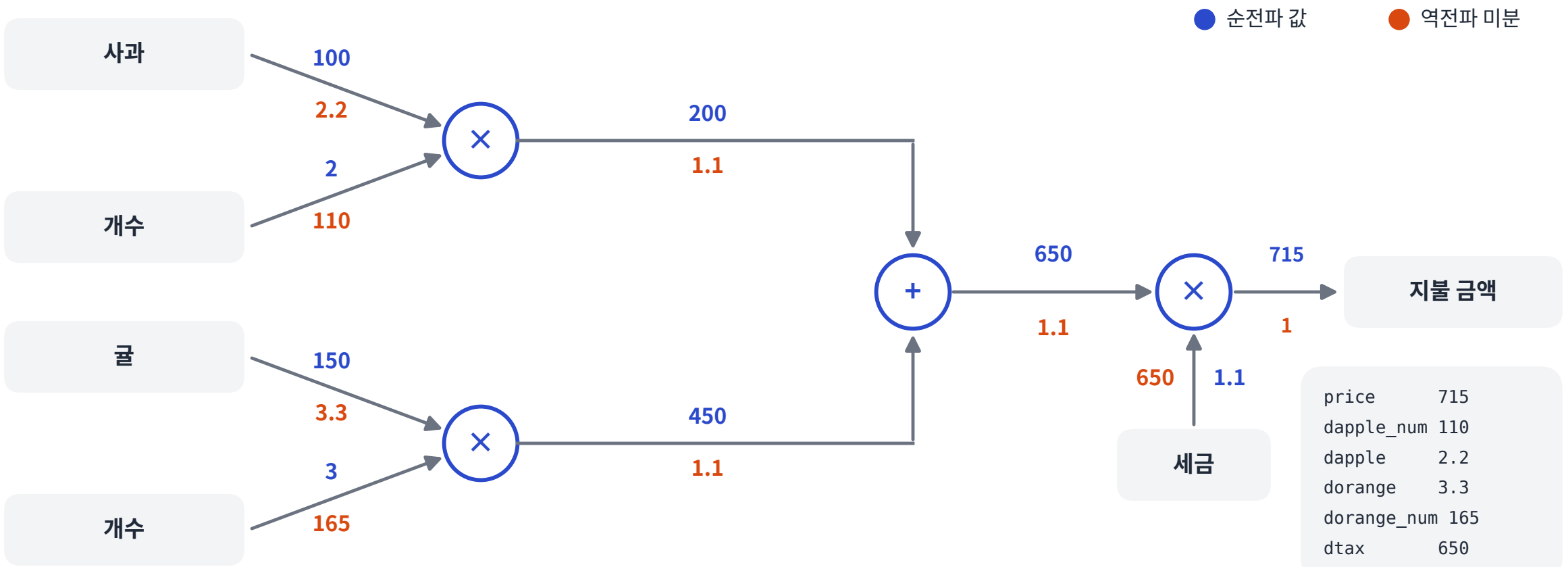


덧셈 노드: dx = dout × 1, dy = dout × 1

	곱셈 계층	덧셈 계층
순전파	x * y	x + y
역전파	입력을 서로 바꿔 곱함	1을 곱해 그대로 전달
__init__	x, y 저장	초기화 불필요

곱셈 + 덧셈 계층: 사과 2개, 귤 3개 구입

사과 100원 × 2, 귤 150원 × 3, 소비세 1.1 → 합계 715원



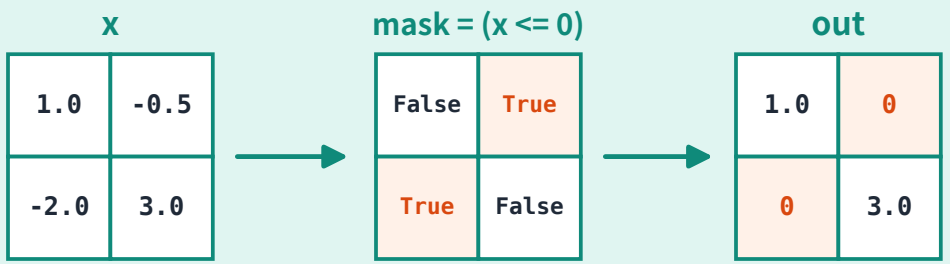
ReLU 계층

순전파 입력이 0 이하인 자리에는 역전파 신호도 보내지 않는다

ReLU 수식

순전파 $y = x \ (x > 0), \quad y = 0 \ (x \le 0)$

역전파 $\frac{\partial y}{\partial x} = 1 \ (x > 0), \quad 0 \ (x \le 0)$



mask가 True인 자리는 순전파·역전파 모두 0

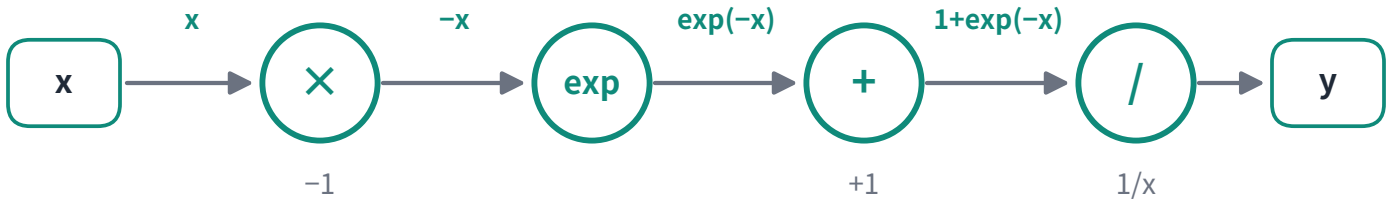
```
class Relu:
    def __init__(self):
        self.mask = None

    def forward(self, x):
        self.mask = (x <= 0)
        out = x.copy()
        out[self.mask] = 0
        return out

    def backward(self, dout):
        dout[self.mask] = 0
        dx = dout
        return dx
```

Sigmoid 계층

$y = \frac{1}{1+\exp(-x)}$ 을 네 개의 노드로 쪼개어 역전파를 따라간다



1단계 '/' 노드

$-\frac{1}{x^2} = -y^2$

상류 값 × ($-y^2$)

2단계 '+' 노드

$\times 1$

상류 값을 그대로 하류로

3단계 'exp' 노드

$\exp(-x)$

상류 값 × exp(−x)

4단계 '×' 노드

$\times(-1)$

순전파 값을 바꿔 곱함 (−1)

$$\frac{\partial L}{\partial y} y^2 e^{-x} = \frac{\partial L}{\partial y} y(1-y)$$

```
class Sigmoid:
    def __init__(self):
        self.out = None

    def forward(self, x):
        out = 1/(1+np.exp(-x))
        self.out = out
        return out

    def backward(self, dout):
        dx = dout * (1.0 - self.out) * self.out
        return dx
```